

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

О Т Ч Е Т

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3.2
«Создание таблиц базы данных PostgreSQL.
Заполнение таблиц рабочими данными»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Смирнов Фёдор Евгеньевич
Факультет прикладной информатики
Группа K3240
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватель Говорова Марина Михайловна

Санкт-Петербург,
2026

Цель работы

Цель работы — овладеть практическими навыками создания таблиц базы данных PostgreSQL, заполнения их рабочими данными, резервного копирования и восстановления базы данных с использованием PostgreSQL и pgAdmin 4. В рамках выполнения лабораторной работы была реализована база данных по предметной области «Прокат автомобилей», разработанной в лабораторной работе № 2.

Практическое задание

В ходе выполнения лабораторной работы требовалось создать базу данных с использованием pgAdmin 4 согласно индивидуальному заданию, создать схему в составе базы данных, создать таблицы базы данных, установить ограничения на данные типов Primary Key, Unique, Check, Foreign Key, заполнить таблицы рабочими данными, создать резервную копию базы данных в двух форматах и выполнить восстановление базы данных. Для варианта 12 предметная область связана с учетом клиентов, автомобилей, договоров проката, продлений, страховок, залога, аварий и штрафов в системе проката автомобилей.

Выполнение задания

1. Наименование БД, вариант, предметная область

Наименование реализованной базы данных: carsharing.

Вариант: 12. БД «Прокат автомобилей».

Предметная область описывает работу компании, предоставляющей автомобили в прокат. В базе данных должны учитываться сведения о клиентах и их паспортах, моделях и конкретных автомобилях, сотрудниках и их должностях, договорах проката, продлениях договоров, страховании, залогах, авариях, нарушениях ПДД и начисленных штрафах. Кроме того, в системе требуется хранить изменяющиеся тарифы аренды и информацию о том, кто оплачивает штраф — клиент или компания.

2. Схема инфологической модели БД ЛР 2 (IDEF1X)

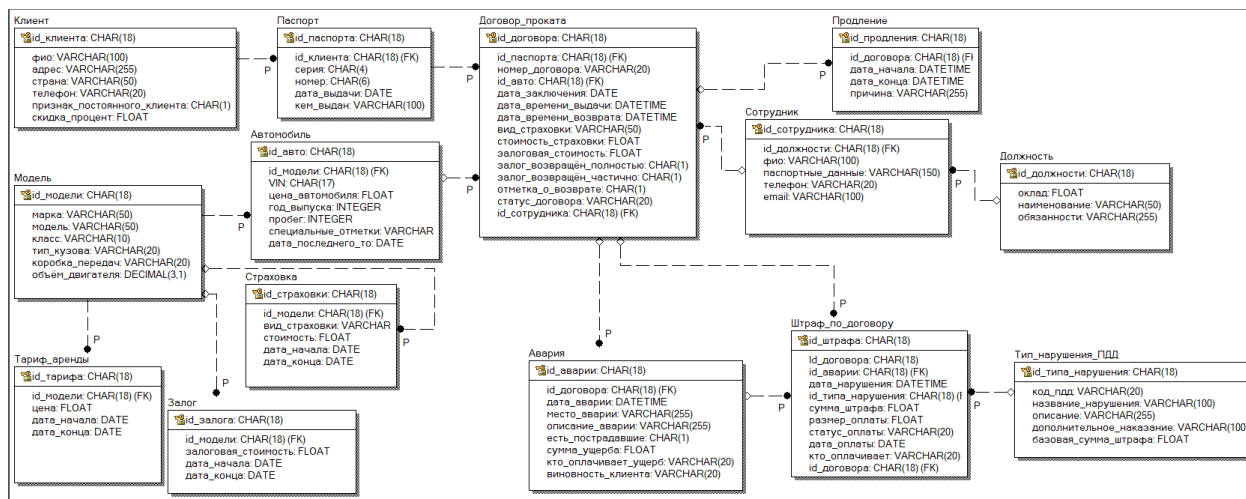


Рисунок 1 — Схема инфологической модели базы данных «Прокат автомобилей» в нотации IDEF1X.

Инфологическая модель базы данных была разработана в лабораторной работе № 2 в соответствии с описанием предметной области варианта 12. В ИЛМ были выделены основные сущности системы: клиент, паспорт, модель автомобиля, автомобиль, тариф аренды, залог, страховка, должность, сотрудник, договор проката, продление, тип нарушения ПДД, авария и штраф по договору. На этапе инфологического моделирования были определены основные атрибуты сущностей, а также связи между ними, отражающие бизнес-логику системы проката автомобилей. Центральной сущностью модели является договор проката, поскольку именно через него связываются клиент, автомобиль, сотрудник, страхование, залог, продление, аварии и штрафы.

3. Схема логической модели БД, сгенерированная в Generate ERD

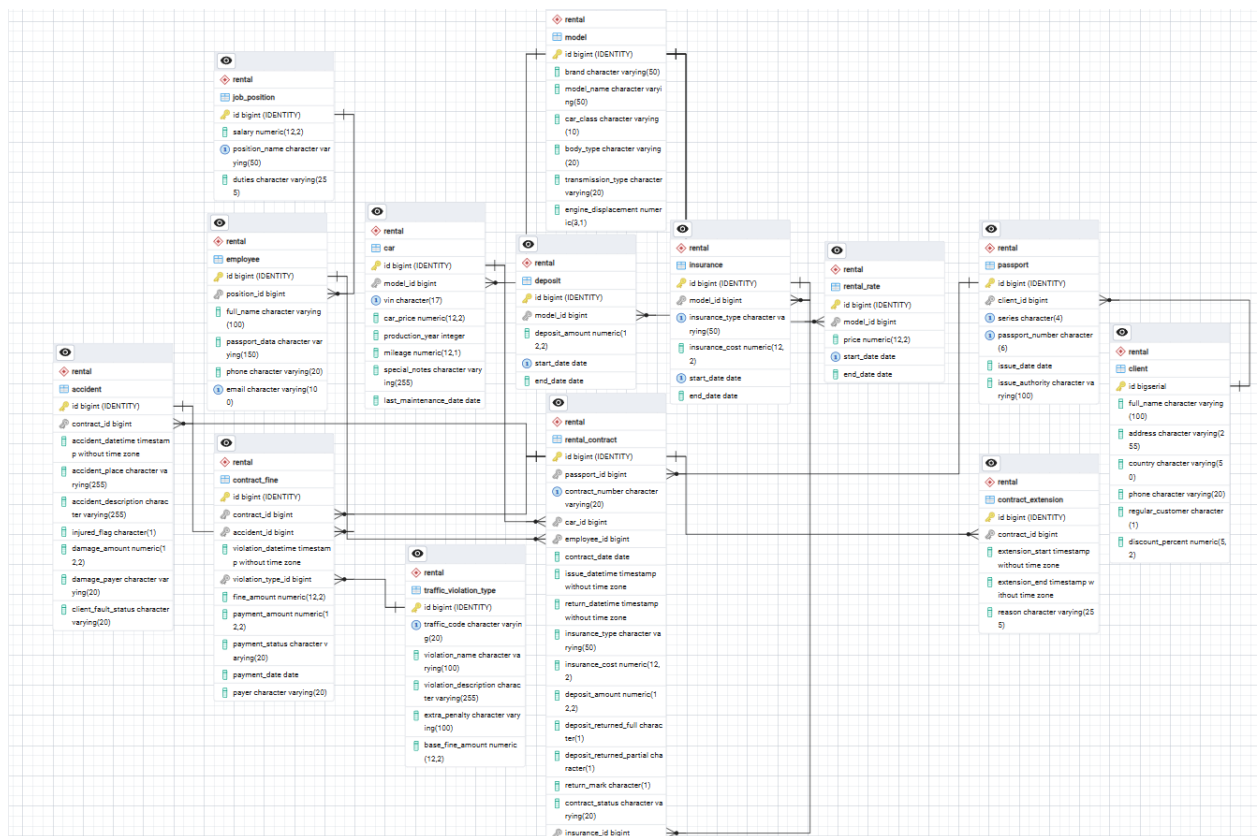


Рисунок 2 — Схема логической модели базы данных, сгенерированная в Generate ERD в pgAdmin 4.

После построения инфологической модели в лабораторной работе № 2 была реализована логическая модель базы данных, а затем в pgAdmin была сформирована схема Generate ERD для уже созданной базы данных PostgreSQL. Полученная ERD-схема отражает фактически реализованные таблицы, их атрибуты и связи, заданные средствами СУБД. В отличие от ИЛМ, логическая модель в pgAdmin показывает не только сущности предметной области, но и физическую реализацию структуры базы данных: таблицы, первичные и внешние ключи, а также связи между ними. Это позволило проверить корректность переноса модели из ЛР 2 в рабочую PostgreSQL-базу.

4. Анализ ключей и замена составных ключей суррогатными

В лабораторной работе № 3.2 отдельно указано, что при реализации базы данных необходимо проанализировать составные ключи сущностей инфологической модели и заменить их суррогатными простыми ключами таблиц. Данное решение было использовано при проектировании физической структуры БД. Для всех основных таблиц были введены отдельные суррогатные первичные ключи типа id, автоматически генерируемые средствами PostgreSQL. Такой подход упростил реализацию внешних ключей, сделал структуру таблиц более удобной для поддержки и позволил избежать использования громоздких составных или естественных идентификаторов в качестве первичных ключей.

Например, для таблиц client, passport, model, car, insurance, rental_contract, accident и других были введены отдельные идентификаторы записей. При этом некоторые естественные идентификаторы не были утрачены, а были сохранены в виде ограничений уникальности. Так, для таблицы паспортов может контролироваться уникальность сочетания серии и номера паспорта, а для таблицы автомобилей — уникальность VIN. Таким образом, суррогатный ключ использовался как основной идентификатор строки, а естественные реквизиты сохранялись как значимые атрибуты предметной области.

5. Реализация таблиц базы данных

В реализованной базе данных были созданы таблицы, соответствующие сущностям предметной области, выделенным в лабораторной работе № 2. Таблица client предназначена для хранения сведений о клиентах, таблица passport — для хранения паспортных данных клиентов, таблица model — для хранения общих характеристик моделей автомобилей, а таблица car — для хранения сведений о конкретных автомобилях компании. Таблицы rental_rate, deposit и insurance используются для учета изменяемых тарифов аренды, залоговой стоимости и страхования в зависимости от модели автомобиля и периода действия условий. Таблицы job_position и employee предназначены для хранения сведений о должностях и сотрудниках. Таблица rental_contract является центральной в системе и содержит сведения о заключенных договорах проката. Таблица contract_extension отражает продления договоров, таблица traffic_violation_type — справочник типов нарушений ПДД, таблица accident — сведения об авариях, а таблица contract_fine — сведения о начисленных штрафах по договорам.

Такое разбиение предметной области на таблицы позволило устранить избыточность хранения данных и отделить справочную информацию от событийных данных. Например, сведения о модели автомобиля хранятся отдельно от сведений о конкретном автомобиле, а тарифы аренды и условия страхования вынесены в самостоятельные таблицы, поскольку в предметной области прямо указано, что цены на прокат автомобилей могут меняться, а стоимость страховки и залоговая стоимость зависят от марки автомобиля.

6. Ограничения целостности

При реализации базы данных были заданы ограничения целостности, предусмотренные лабораторной работой № 3.2: Primary Key, Unique, Check, Foreign Key. Первичные ключи использованы для однозначной идентификации записей во всех основных таблицах. Ограничения уникальности были заданы для атрибутов, которые по смыслу не должны повторяться, например для VIN автомобиля и для отдельных комбинаций реквизитов, идентифицирующих документы или записи. Ограничения типа Check использовались для контроля допустимых значений, например для

признаков типа Y/N, диапазонов числовых значений, допустимых дат и других условий предметной области. Ограничения внешних ключей обеспечили ссылочную целостность между таблицами и гарантировали корректность ссылок между клиентами и паспортами, моделями и автомобилями, договорами и автомобилями, договорами и сотрудниками, договорами и страховками, договорами и штрафами.

Задание ограничений целостности позволило перенести в PostgreSQL не только структуру данных, но и ключевые бизнес-правила предметной области. В частности, были реализованы связи между договором и конкретным автомобилем, между договором и выбранной страховкой, между штрафами и типами нарушений ПДД, а также между авариями и договорами проката. Это позволяет не только хранить данные, но и контролировать их корректность на уровне самой СУБД.

7. Заполнение таблиц рабочими данными

1 `SELECT * FROM rental.client;`

Data Output Messages Notifications

Showing rows: 1 to 3

	id [PK] bigint	full_name character varying (100)	address character varying (255)	country character varying (50)	phone character varying (20)	regular_customer character (1)	discount_percent numeric (5,2)
1	1	Иванов Иван Сергеевич	Санкт-Петербург, Невский пр., д. 10, кв. 5	Россия	+7-911-111-11-11	Y	10.00
2	2	Петрова Анна Викторовна	Санкт-Петербург, ул. Типанова, д. 14, кв. ...	Россия	+7-921-222-22-22	N	0.00
3	3	Смирнов Дмитрий Олегов...	Санкт-Петербург, Ленинский пр., д. 45, к...	Россия	+7-931-333-33-33	Y	5.00

Рисунок 3 — Результат просмотра содержимого таблицы `client` после заполнения рабочими данными.

1 `SELECT * FROM rental.rental_contract;`

Data Output Messages Notifications

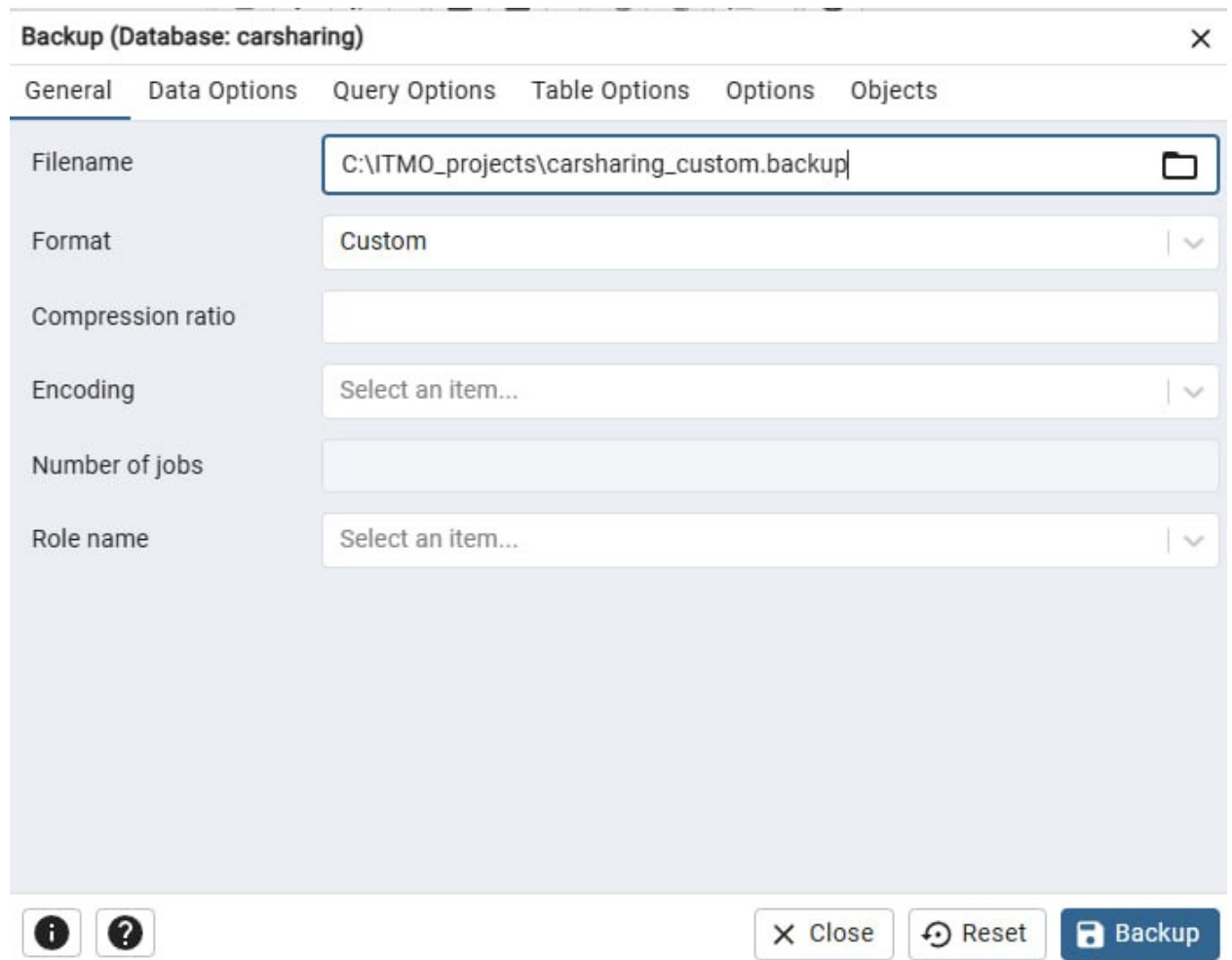
Showing rows: 1 to 3 Page No: 1 of 1

	id [PK] bigint	passport_id bigint	contract_number character varying (20)	car_id bigint	employee_id bigint	contract_date date	issue_datetime timestamp without time zone	return_datetime timestamp without time zone	insurance_type character varying (50)	insurance_cost numeric (12,2)	deposit_amount numeric (12,2)	deposit_returned_full character (1)
1	1	1	RC-2025-001	1	1	2025-03-10	2025-03-10 10:00:00	2025-03-15 18:00:00	КАСКО базовое	2500.00	15000.00	Y
2	2	2	RC-2025-002	2	2	2025-04-01	2025-04-01 09:30:00	2025-04-05 20:00:00	Полное КАСКО	3200.00	20000.00	N
3	3	3	RC-2025-003	3	1	2025-05-20	2025-05-20 11:00:00	[null]	ОСАГО +	1800.00	12000.00	N

Рисунок 4 — Результат просмотра содержимого таблицы `rental_contract` после заполнения рабочими данными.

Для проверки корректности заполнения базы данных после выполнения запросов INSERT были выполнены запросы SELECT к заполненным таблицам. На рисунках представлены результаты просмотра содержимого таблиц `client` и `rental_contract`. Полученные результаты подтверждают, что данные были успешно загружены в базу, а созданные ранее связи и ограничения целостности не препятствуют корректной работе с данными. Таблица `client` содержит сведения о клиентах системы проката автомобилей, а таблица `rental_contract` отражает сведения о заключённых договорах проката, включая ссылки на паспорт клиента, автомобиль, сотрудника и параметры договора.

8. Резервное копирование БД



Backup (Database: carsharing)

General Data Options Query Options Table Options Options Objects

Filename C:\ITMO_projects\carsharing_custom.backup

Format Custom

Compression ratio

Encoding Select an item...

Number of jobs

Role name Select an item...

Close Reset Backup

Рисунок 5 — Окно создания резервной копии базы данных *carsharing* в формате *Custom* в *pgAdmin 4*.

Согласно требованиям лабораторной работы были созданы две резервные копии базы данных. Первая копия была создана в формате *Custom* и предназначена для последующего восстановления базы данных. Вторая копия была создана в формате *Plain* в виде текстового SQL-скрипта, пригодного для размещения в отчете в качестве листинга. При создании резервных копий в *pgAdmin* использовались параметры *Dump options*, в том числе параметры групп *Type of objects* и *Queries*, как это указано в методических материалах. Формат *Plain* был использован для получения редактируемого текстового сценария, содержащего SQL-команды, а формат *Custom* — для последующего восстановления базы данных с помощью *pg_restore* / диалога *Restore*.

9. Восстановление БД

Рисунок 6 — Окно восстановления базы данных из резервной копии в формате Custom в pgAdmin 4.

После создания резервной копии было выполнено восстановление базы данных из файла формата Custom. Для восстановления была подготовлена база данных-приёмник, после чего в диалоге Restore были заданы основные параметры: формат файла резервной копии, путь к файлу, число заданий и роль пользователя. Восстановление позволило убедиться в том, что созданная база данных может быть воспроизведена из резервной копии без потери структуры таблиц, связей и загруженных данных.

10. Dump, содержащий скрипты работы с БД

В качестве листинга реализации базы данных в отчёт включены фрагменты текстового dump-файла в формате Plain. Данный dump содержит команду создания базы данных, создание схемы, операторы создания таблиц, определения ограничений целостности, включая внешние ключи, а также запросы на вставку данных. В листингах ниже приведены наиболее показательные фрагменты файла: создание базы данных и схемы, создание центральной таблицы rental_contract, определение внешних ключей и заполнение таблиц рабочими данными.

Создание базы данных и схемы

```
--  
-- PostgreSQL database dump  
--  
  
CREATE DATABASE carsharing WITH TEMPLATE = template0 ENCODING = 'UTF8'  
LOCALE_PROVIDER = libc LOCALE = 'en';  
  
ALTER DATABASE carsharing OWNER TO postgres;
```



```

\connect carsharing

CREATE SCHEMA rental;

ALTER SCHEMA rental OWNER TO postgres;

```

Создание центральной таблицы rental_contract

```

CREATE TABLE rental.rental_contract (
    id bigint NOT NULL,
    passport_id bigint NOT NULL,
    contract_number character varying(20) NOT NULL,
    car_id bigint NOT NULL,
    employee_id bigint NOT NULL,
    contract_date date NOT NULL,
    issue_datetime timestamp without time zone NOT NULL,
    return_datetime timestamp without time zone,
    insurance_type character varying(50),
    insurance_cost numeric(12,2),
    deposit_amount numeric(12,2) NOT NULL,
    deposit_returned_full character(1) DEFAULT 'N'::bpchar,
    deposit_returned_partial character(1) DEFAULT 'N'::bpchar,
    return_mark character(1) DEFAULT 'N'::bpchar,
    contract_status character varying(20) NOT NULL,
    insurance_id bigint,
    CONSTRAINT chk_rental_contract_deposit_amount CHECK ((deposit_amount > (0)::numeric)),
    CONSTRAINT chk_rental_contract_deposit_returned_full CHECK ((deposit_returned_full = ANY
(ARRAY['Y'::bpchar, 'N'::bpchar])),
    CONSTRAINT chk_rental_contract_deposit_returned_partial CHECK ((deposit_returned_partial =
ANY (ARRAY['Y'::bpchar, 'N'::bpchar])),
    CONSTRAINT chk_rental_contract_insurance_cost CHECK (((insurance_cost IS NULL) OR
(insurance_cost >= (0)::numeric))),
    CONSTRAINT chk_rental_contract_number_not_empty CHECK ((btrim((contract_number)::text)
<> ''::text)),
    CONSTRAINT chk_rental_contract_return_flags CHECK ((NOT ((deposit_returned_full =
'Y'::bpchar) AND (deposit_returned_partial = 'Y'::bpchar))),
    CONSTRAINT chk_rental_contract_return_mark CHECK ((return_mark = ANY
(ARRAY['Y'::bpchar, 'N'::bpchar])),
    CONSTRAINT chk_rental_contract_return_time CHECK (((return_datetime IS NULL) OR
(return_datetime >= issue_datetime))),
    CONSTRAINT chk_rental_contract_status CHECK (((contract_status)::text = ANY
(ARRAY[(('открыт'::character varying)::text, ('закрыт'::character varying)::text,
('просрочен'::character varying)::text, ('аннулирован'::character varying)::text]))
);

```

Внешние ключи

```

ALTER TABLE ONLY rental.passport
    ADD CONSTRAINT fk_passport_client
        FOREIGN KEY (client_id)
        REFERENCES rental.client(id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT;

ALTER TABLE ONLY rental.rental_contract
    ADD CONSTRAINT fk_rental_contract_car
        FOREIGN KEY (car_id)
        REFERENCES rental.car(id)
        ON UPDATE CASCADE

```

```
ON DELETE RESTRICT;
```

```
ALTER TABLE ONLY rental.rental_contract  
ADD CONSTRAINT fk_rental_contract_employee  
FOREIGN KEY (employee_id)  
REFERENCES rental.employee(id)  
ON UPDATE CASCADE  
ON DELETE RESTRICT;
```

```
ALTER TABLE ONLY rental.rental_contract  
ADD CONSTRAINT fk_rental_contract_insurance  
FOREIGN KEY (insurance_id)  
REFERENCES rental.insurance(id)  
ON UPDATE CASCADE  
ON DELETE RESTRICT;
```

```
ALTER TABLE ONLY rental.rental_contract  
ADD CONSTRAINT fk_rental_contract_passport  
FOREIGN KEY (passport_id)  
REFERENCES rental.passport(id)  
ON UPDATE CASCADE  
ON DELETE RESTRICT;
```

Вставка данных

```
INSERT INTO rental.client VALUES
```

```
(1, 'Иванов Иван Сергеевич', 'Санкт-Петербург, Невский пр., д. 10, кв. 5', 'Россия', '+7-911-111-11-11', 'Y', 10.00);
```

```
INSERT INTO rental.client VALUES
```

```
(2, 'Петрова Анна Викторовна', 'Санкт-Петербург, ул. Типанова, д. 14, кв. 21', 'Россия', '+7-921-222-22-22', 'N', 0.00);
```

```
INSERT INTO rental.client VALUES
```

```
(3, 'Смирнов Дмитрий Олегович', 'Санкт-Петербург, Ленинский пр., д. 45, кв. 8', 'Россия', '+7-931-333-33-33', 'Y', 5.00);
```

```
INSERT INTO rental.rental_contract VALUES
```

```
(1, 1, 'RC-2025-001', 1, 1, '2025-03-10', '2025-03-10 10:00:00', '2025-03-15 18:00:00', 'КАСКО базовое', 2500.00, 15000.00, 'Y', 'N', 'Y', 'закрыт', 1);
```

```
INSERT INTO rental.rental_contract VALUES
```

```
(2, 2, 'RC-2025-002', 2, 2, '2025-04-01', '2025-04-01 09:30:00', '2025-04-05 20:00:00', 'Полное КАСКО', 3200.00, 20000.00, 'N', 'Y', 'Y', 'закрыт', 2);
```

```
INSERT INTO rental.rental_contract VALUES
```

```
(3, 3, 'RC-2025-003', 3, 1, '2025-05-20', '2025-05-20 11:00:00', NULL, 'ОСАГО +', 1800.00, 12000.00, 'N', 'N', 'N', 'открыт', 3);
```

Выводы

В ходе выполнения лабораторной работы была реализована база данных PostgreSQL по предметной области «Прокат автомобилей». На основе модели, разработанной в лабораторной работе № 2, была создана рабочая структура БД в pgAdmin, включающая схему, таблицы, ключи, ограничения целостности и тестовые данные. Также были освоены практические операции резервного копирования базы данных в форматах Custom и Plain, а затем выполнено восстановление базы данных из резервной копии. Таким образом, были получены практические навыки физической реализации модели данных в

PostgreSQL, контроля целостности данных и выполнения операций обслуживания базы данных.

Реализованная база данных отражает основные сущности и связи предметной области проката автомобилей и может использоваться как основа для дальнейшего развития системы. В ходе работы была подтверждена важность предварительного инфологического и логического моделирования, поскольку именно результаты лабораторной работы № 2 послужили основой для физической реализации базы данных и настройки ограничений целостности в PostgreSQL.